Manual de Despliegue en la Nube - API de Detección de Fraude (Teorico).

1. Objetivo y estrategia de despliegue. Este manual describe una práctica de despliegue en la nube de una API de detección de fraude desarrollada con FastAPI y contenerizada con Docker. Se utilizará AWS Elastic Beanstalk como plataforma como servicio (PaaS) en una nube pública. El proveedor automatiza el aprovisionamiento y la gestión del entorno; el responsable del proyecto conserva la responsabilidad sobre el código, el modelo, las dependencias, los permisos y la configuración de seguridad. Para mantener el entorno del contenedor, se seleccionará la plataforma Docker, no Python. El flujo será:

cliente o frontend → API REST → validación de entradas → modelo model.pkl → respuesta JSON. El entrenamiento queda separado del servicio de inferencia. Esta guía no acredita un despliegue ejecutado ni habilita decisiones financieras automáticas: las pruebas deben realizarse con datos sintéticos y las predicciones se consideran demostrativas.

2. Requerimientos técnicos y condiciones previas. Se requiere una cuenta activa de AWS, una región definida, permisos para crear el entorno de Elastic Beanstalk y sus recursos, un rol de servicio y un perfil de instancia EC2 con permisos adecuados. También se necesita un repositorio de GitHub con el código fuente, Git instalado para versionarlo y Docker en funcionamiento para validar la imagen localmente. GitHub sirve como control de versiones; subir el código al repositorio no lo despliega automáticamente en AWS. El proyecto debe incluir main.py, model.pkl, requirements.txt y Dockerfile. Las dependencias incluyen FastAPI, Uvicorn, scikit-learn y Pandas cuando el código lo utilice; se agregará joblib si ese fue el método de serialización. Las versiones de Python y de las bibliotecas deberán coincidir con el entorno validado del modelo. Solo se cargarán archivos de modelo de origen confiable, pues pickle y joblib pueden ejecutar código al deserializar.

3. Procedimiento paso a paso: práctica de despliegue en AWS. Ejecutar las siguientes etapas en orden. Conservar evidencia de cada validación y no registrar como exitoso un paso que solo se haya descrito. Si se realiza únicamente una simulación sin crear recursos, marcar los resultados como pendientes.

1. **Preparar y versionar el proyecto.** Verificar que main.py exponga la aplicación como app y cargue model.pkl desde una ruta válida dentro del contenedor. Mantener el orden de las variables y el preprocesamiento utilizados en el entrenamiento; si el modelo se entrenó con columnas de un DataFrame, conservar sus nombres. Documentar el contrato de entrada y salida de la API y añadir una ruta de salud, por ejemplo GET /health, que devuelva 200 solo cuando el servicio

esté listo y el modelo se haya cargado. Subir main.py, model.pkl, requirements.txt y Dockerfile a GitHub únicamente si se cuenta con autorización para compartir el modelo. Excluir credenciales, archivos .env, datos reales, el entorno virtual y archivos temporales. Registrar el identificador del commit y la versión del modelo.

2. **Revisar el Dockerfile.** Mantener la imagen base y las versiones que ya funcionaron en las pruebas locales; no sustituirlas por versiones arbitrarias. Comprobar que el archivo defina el directorio de trabajo, copie requirements.txt, instale sus dependencias y copie main.py y model.pkl. Para el puerto acordado en esta guía, las instrucciones finales deben ser:
   EXPOSE 8000
   CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
   El comando se configura dentro del Dockerfile, no como un inicio de la plataforma Python de Elastic Beanstalk. EXPOSE declara el puerto del contenedor; no abre por sí mismo un puerto público. Utilizar un archivo .dockerignore para excluir secretos y archivos innecesarios del contexto de construcción.
3. **Validar el contenedor localmente.** Desde la carpeta del proyecto, ejecutar:
   docker build -t fraude-api:local .
   docker run --rm --name fraude-api -p 127.0.0.1:8000:8000 fraude-api:local
   Mantener esa terminal abierta. En otra terminal, comprobar la ruta de salud y consultar /docs si la documentación interactiva de FastAPI está habilitada. En Windows puede utilizarse:
   curl.exe -i http://localhost:8080/health
   La respuesta esperada de la ruta de salud es HTTP 200, siempre que se haya implementado. Probar el endpoint real de inferencia con solicitudes válidas e inválidas y registrar el JSON recibido. Comparar las predicciones con las obtenidas localmente para las mismas entradas; incluir casos límite del material de despliegue local, como [10, 12, 0], [999, 1, 1] y [500, 23, 0], solo si el contrato del modelo utiliza monto, hora y país riesgoso en ese orden. No asumir probabilidades ni etiquetas sin ejecutar el modelo. Detener la prueba con Ctrl+C.
4. **Crear el paquete de despliegue.** Generar un archivo ZIP con main.py, model.pkl, requirements.txt, Dockerfile y cualquier módulo o recurso requerido por la aplicación. El Dockerfile debe quedar en la raíz del ZIP, no dentro de una carpeta contenedora. Abrir el archivo y comprobar su estructura antes de subirlo. Este procedimiento utiliza el código y el Dockerfile para que Elastic Beanstalk construya la imagen; no requiere publicar previamente la imagen en un registro. Desplegar una imagen ya publicada sería una alternativa distinta que necesitaría su configuración correspondiente.

5. **Acceder a AWS y crear la aplicación.** Iniciar sesión con una identidad autorizada, seleccionar la región acordada y abrir Elastic Beanstalk. Crear una aplicación, por ejemplo fraude-api, y un entorno de servidor web para la práctica. Registrar el nombre de la aplicación, el entorno y la región. No utilizar credenciales de la cuenta raíz como práctica habitual ni incluir claves de acceso en el código.
6. **Seleccionar plataforma y cargar la versión.** Elegir la plataforma Docker y una rama soportada, como Docker sobre Amazon Linux 2023 cuando esté disponible. Utilizar el entorno estándar basado en EC2 para este procedimiento, no Docker administrado por ECS ni un entorno de clúster. Seleccionar la opción para cargar código propio, subir el ZIP y asignar una etiqueta de versión trazable al commit. No seleccionar Python: aunque la API está escrita en Python, su entorno de ejecución se define mediante Docker.
7. **Configurar acceso, capacidad y salud.** Asignar el rol de servicio de Elastic Beanstalk y el perfil de instancia EC2. Si no existe este último, crearlo en IAM con los permisos necesarios para el entorno web y seleccionarlo en el asistente. Revisar la VPC, las subredes y los grupos de seguridad con permisos mínimos. Para una demostración de baja carga, una instancia única puede ser suficiente, pero no proporciona alta disponibilidad. Si se requiere balanceo y escalabilidad, elegir un entorno con balanceador y límites de capacidad explícitos, asumiendo sus costos. Mantener Uvicorn escuchando en 0.0.0.0:8000 dentro del contenedor y EXPOSE 8000 en el Dockerfile para el flujo sin Docker Compose. No abrir el puerto 8000 directamente a internet. En un entorno con balanceador, configurar la comprobación de salud hacia /health si esa ruta existe y responde 200; en una instancia única verificarla también desde el cliente. Añadir variables de configuración sin incrustar secretos en el paquete.
8. **Desplegar y verificar la API.** Revisar la configuración y crear el entorno. Esperar a que termine la construcción y examinar los eventos y el estado de salud. Copiar la dirección asignada por Elastic Beanstalk y registrar su valor real, sin inventar una URL. Comprobar /health y el endpoint de inferencia documentado en main.py. Si /docs está habilitado, usar su esquema para enviar una solicitud; no suponer que la ruta se llama /predict ni que los campos tienen nombres específicos. Confirmar que las entradas válidas reciben el resultado esperado y que las inválidas producen una respuesta controlada según el contrato. Comparar la ejecución local y la remota con el mismo modelo y los mismos datos. Guardar fecha, versión, código HTTP y respuesta, sin datos confidenciales. Un entorno creado no equivale a un modelo validado: ambos controles deben quedar documentados.

9. **Asegurar y operar el servicio.** Antes de enviar datos reales o información de acceso, configurar HTTPS y un mecanismo de autenticación y autorización en la API o en su capa de acceso. En entornos con Application Load Balancer, asignar un certificado válido para el dominio mediante ACM y configurar un listener HTTPS en el puerto 443. Una instancia única no dispone de balanceador y requiere otra configuración de terminación TLS; no asumir que su dirección ya ofrece HTTPS. Si existe un frontend en otro origen, limitar CORS a los orígenes autorizados; CORS no sustituye la autenticación. Revisar eventos y registros del entorno y, cuando se configure CloudWatch, observar errores, latencia y consumo de recursos. No registrar credenciales ni transacciones reales. El frontend es un cliente opcional y no se despliega como parte de esta guía.
10. **Resolver fallos, actualizar y cerrar la práctica.** Si aparece un error 502 o 503, revisar primero los registros, el arranque de Uvicorn, el puerto y la ruta de salud; estos códigos no identifican por sí solos la causa. Ante fallos de carga del modelo, comprobar la ruta, el método de serialización y las versiones de las bibliotecas. Ante errores de construcción, revisar requirements.txt y los archivos del ZIP; ante errores de acceso a recursos, revisar los roles IAM sin conceder permisos indiscriminados. Para actualizar, cargar una nueva versión etiquetada y repetir las pruebas. Si la validación falla, volver a desplegar una versión anterior conocida y verificarla; esto no revierte cambios externos de datos. Al terminar la demostración, terminar el entorno de Elastic Beanstalk y revisar por separado los recursos persistentes, compartidos o creados fuera del entorno, así como almacenamiento, registros y facturación, para evitar cargos innecesarios.

4. Documentación técnica, apoyo de IA y criterios de aceptación. La estructuración de este manual se apoyó en inteligencia artificial generativa.

Despliegue en la nube. La IA puede apoyar la redacción del Dockerfile y la estandarización del código, pero los archivos reales y su ejecución requieren revisión humana; aquí no se afirma que hayan sido modificados o probados. Se toma ISO/IEC 23053:2022 como referencia conceptual para describir la tarea, los datos, el modelo, sus componentes y sus etapas de uso; esto no representa una certificación ni una declaración de cumplimiento normativo. La bitácora debe conservar propósito y alcance, versiones de Python y dependencias, procedencia del modelo, variables de entrada y preprocesamiento, commit, configuración del entorno, pruebas locales y remotas, limitaciones, mecanismos de monitoreo y procedimiento de reversión y cierre. El despliegue se aceptará únicamente cuando el contenedor arranque, el modelo cargue, la API responda conforme a su contrato, las pruebas válidas e inválidas estén documentadas y la configuración de seguridad sea apropiada para su uso. Quedan pendientes de

completar con evidencia real: repositorio, commit, versión del modelo, región, entorno, URL, contrato de inferencia y resultados de las pruebas. Referencias consultadas: materiales académicos compartidos; documentación oficial de AWS sobre despliegue Docker, paquetes de aplicación, perfiles de instancia, HTTPS y precios; guía de persistencia de modelos de scikit-learn; y descripción oficial de ISO/IEC 23053:2022.